PARUL UNIVERSITY

FACULTY OF IT & COMPUTER SCIENCE

ACADEMIC YEAR 2025 -26

5. Write prolog programs for following problems

Ans:

Implement a single-player game using a heuristic function to guide the player towards the goal.

```
% Define the initial state and goal state
initial_state(state(0, 0)).
goal_state(state(4, 4)).

% Heuristic function to evaluate the state
heuristic(state(X, Y), H) :-
    goal_state(state(GX, GY)),
    H is abs(GX - X) + abs(GY - Y).

% Define possible moves
move(state(X, Y), state(X1, Y1)) :-
    (X1 is X + 1, Y1 is Y);
    (X1 is X - 1, Y1 is Y);
    (X1 is X, Y1 is Y + 1);
    (X1 is X, Y1 is Y - 1).

% Game loop to reach the goal state
play(State) :-
    goal_state(State),
    write('Goal reached!'), nl.
play(State) :-
    findall(NextState, move(State, NextState), Moves),
```

```
    maplist(heuristic, Moves, Heuristics),
    min_list(Heuristics, MinH),
    nth0(Index, Heuristics, MinH),
    nth0(Index, Moves, NextState),
    play(NextState).

% Start the game
start :-
    initial_state(State),
    play(State).
```

6. Demonstrate Knowledge Base and Query System in prolog

Ans:

```
% Facts
parent(john, mary).
parent(mary, susan).
parent(mary, tom).
parent(susan, alice).

% Rules
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.
```

Find all children of Mary: ?- parent(mary, Child).

Find all grandparents of Alice: ?- grandparent(Grandparent, alice).

Find all siblings of Susan: ?- sibling(susan, Sibling).

7. Convert Prolog predicates into Semantic Net

Ans:

```
% Facts
node(john).
node(mary).
node(susan).
```

```
node(tom).
node(alice).

edge(parent, john, mary).
edge(parent, mary, susan).
edge(parent, mary, tom).
edge(parent, susan, alice).

% Rules to query the Semantic Net
is_parent(X, Y) :- edge(parent, X, Y).

?- is_parent(mary, susan).
true.
?- is_parent(john, alice).
false.
```

8. Demonstrate supervised learning using artificial neural network

Ans:

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Generate a random dataset for binary classification
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

class NeuralNetwork:
    def _init_(self, input_size, hidden_size, output_size):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size
```

9. Write a program to implement Travelling Salesman Problem in prolog

Ans:

```
% Define the distances between cities
distance(a, b, 10).
distance(a, c, 15).
distance(a, d, 20).
distance(b, c, 35).
distance(b, d, 25).
distance(c, d, 30).

% Ensure the distance is symmetric
distance(X, Y, D) :- distance(Y, X, D).

% Find all permutations of a list
permute([], []).
permute(L, [H|T]) :-
```

```
    select(H, L, R),
    permute(R, T).

% Calculate the total distance of a route
route_distance([_], 0).
route_distance([A, B|T], D) :-
    distance(A, B, D1),
    route_distance([B|T], D2),
    D is D1 + D2.

% Find the shortest route
tsp(Start, Path, MinDist) :-
    findall(P, permute([a, b, c, d], P), Perms),
    findall(D, (member([Start|T], Perms), append([Start|T], [Start], Path), route_distance(Path, D)), Dists),
    min_list(Dists, MinDist).

% Query to find the shortest route starting from city 'a'
?- tsp(a, Path, MinDist).
```